

Agent 子循环与 Python 参数机制复习笔记

整理范围：本次对话全部内容 | 主题：子 agent、消息结构、dict、函数参数、**kwargs

核心总览：这次对话其实围绕一条主线展开：Agent 系统如何把 LLM 返回的结构化消息，转化为 Python 里的函数调用与结果回填。

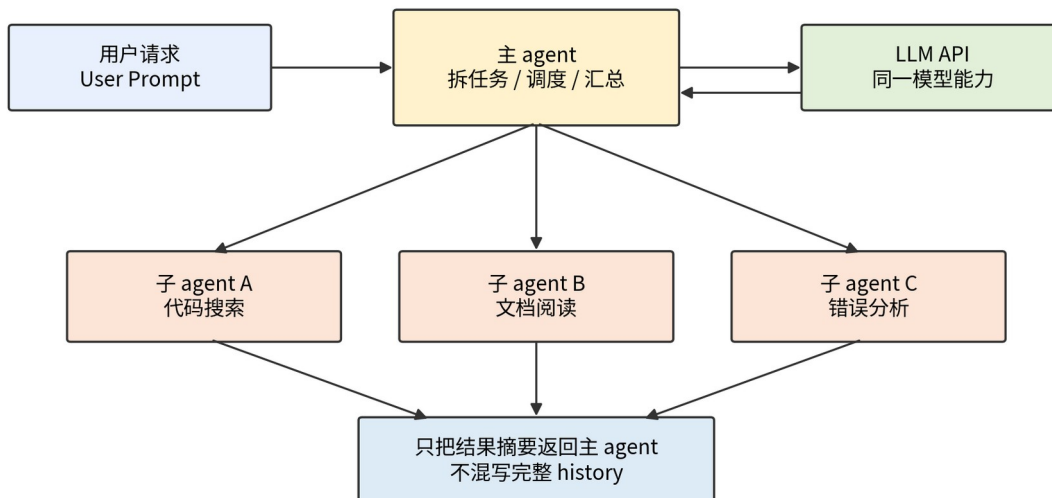
目录

- 1. 主 agent / 子 agent 与 LLM API 调用关系
- 2. 子 agent 是否允许继续派遣 agent
- 3. message / content / block 的数据结构
- 4. extract_text 与子 agent 结果收敛逻辑
- 5. Python for 循环、list、dict 的遍历与取值
- 6. TOOL_HANDLER 字典与工具分发表
- 7. 函数参数体系：形参、实参、位置参数、关键字参数等
- 8. handler(**block.input) 的本质
- 9. 重点易错点与最佳实践

1. 主 agent / 子 agent 与同一个 LLM API

重点：主 agent 和子 agent 可以共用同一个 LLM API，但不能共用同一个消息历史。共用的是模型访问能力，不是共用一个对话状态。

概念	正确理解
同一个 LLM API	可以是同一个 base_url、model、API key、client 配置。
独立 message history	主 agent 有 main_history，子 agent 有 sub_history，彼此隔离。
并发 concurrent	多个 API 请求在时间上重叠发出，这是调用方需要关注的。
并行 parallel	是否真的同时在多 GPU/多服务器上推理，是供应商内部实现，调用方一般不用关心。
平台限制	需要遵守 RPM、TPM、并发连接数、余额等限制，可能触发 429/rate limit。



架构记忆：主 agent 负责拆解、调度、汇总；子 agent 负责完成明确的局部任务；子 agent 完成后只把摘要返回主 agent。

2. 子 agent 是否应该再派遣 agent

结论：学习阶段和中小型项目里，默认限制子 agent 独立完成任务，不建议允许它无限再派遣。复杂项目可以做“受控再派遣”。

方案	适合阶段	优点	风险
子 agent 独立完成	学习阶段 / 中小项目	稳定、好调试、成本可控	任务拆解能力有限
子 agent 可继续派遣	大型复杂任务	分工更细、并发更强	调用成本、上下文、任务边界容易失控
受控再派遣	工程化系统	能力与稳定性折中	需要 max_depth、max_children、budget、timeout 等限制

建议实现顺序：先做 Level 0 主 agent + Level 1 子 agent/工具，禁止 Level 2；等主循环、工具调用、hook、permission、todo 同步都稳定后，再考虑开放受控多级 agent。

3. message / content / block 的数据结构

在 agent loop 里，messages 通常是一个 list，里面每个元素是一个 message dict。message dict 通常包含 role 和 content。content 有时是字符串，有时是 block 列表。

```
messages = [
  {"role": "user", "content": "Read README.md"},
  {
    "role": "assistant",
```

```

"content": [
    TextBlock(type="text", text="我需要读取文件"),
    ToolUseBlock(type="tool_use", name="read_file", input={"path": "README.md"})
]
}
]

```

层级	数据类型	常见字段	说明
messages	list	多个 message	整个对话历史。
message	dict	role, content	一条消息。
content	str 或 list	文本或 blocks	assistant 返回时常是 block 列表。
TextBlock	对象	type, text	普通文本块。
ToolUseBlock	对象	type, name, input	工具调用块。

4. extract_text: 从 content blocks 里提取纯文本

```

def extract_text(content) -> str:
    """Extract text from message content blocks."""
    if not isinstance(content, list):
        return str(content)
    return "
.join(getattr(b, "text", "") for b in content if getattr(b, "type", None) == "text")

```

作用：如果 content 不是 list，就直接转成字符串；如果 content 是 block 列表，就只取 type == "text" 的 block，并把它们的 text 用换行拼接。

代码片段	作用
isinstance(content, list)	判断 content 是否是 list。
getattr(b, "type", None)	安全获取 b.type；没有 type 时返回 None。
getattr(b, "text", "")	安全获取 b.text；没有 text 时返回空字符串。
"\n".join(...)	把多个文本片段用换行连接成一个字符串。

注意：这里对 block 用 getattr，是因为 block 可能是对象，如 TextBlock(type=..., text=...)；而不是 dict。

5. messages[-1]["content"] 与子 agent 结果收敛

```

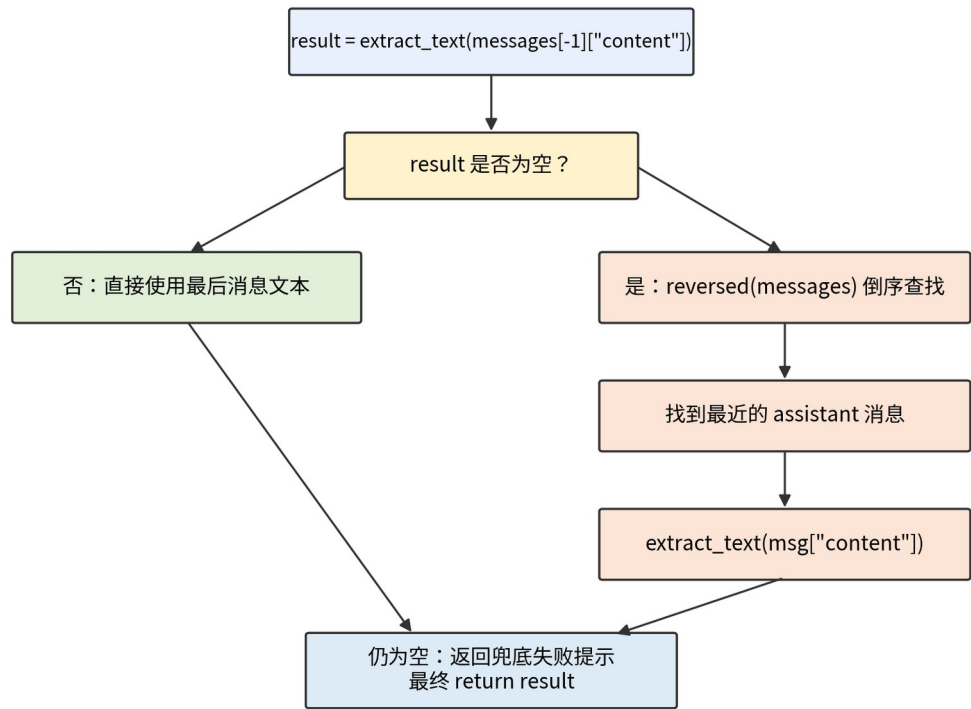
result = extract_text(messages[-1]["content"])

if not result:
    # last message is tool_result, look backwards for assistant text
    for msg in reversed(messages):
        if msg["role"] == "assistant":
            result = extract_text(msg["content"])
            if result:
                break

```

```
if not result:
    result = "Subagent stopped after 30 turns without final answer."

print(f"\x1b[35m[Subagent done]\x1b[0m")
return result
```



重点：子 agent 最后一条消息可能是 tool_result，不一定是 assistant 文本。所以如果最后一条提取不到文本，就倒序查找最近一条 assistant 文本。

变量名易错：前面如果写 results = extract_text(...)，后面却 return result，可能导致 result 未定义。推荐统一使用 result。

6. Python for 循环、list、dict 的遍历与取值

准确说法：Python 的 for 循环不是按“schema 的最小单位”自动理解，而是按可迭代对象每次产出的元素来遍历。

对象类型	for 每次拿到什么	例子
list	列表里的一个元素	for msg in messages: 每次拿到一个 message dict
dict	默认拿到 key	for k in msg: 依次得到 "role", "content"
str	一个字符	for ch in "abc": 得到 a, b, c
tuple	元组里的一个元素	for x in (1,2): 得到 1, 2

```
messages = [
    {"role": "user", "content": "你好"},
    {"role": "assistant", "content": "你好，我可以帮你。"}
]

for msg in messages:
    print(msg["role"], msg["content"])
```

msg 是 dict，所以 msg["role"] 是按 key 取 value。[] 不只给 list 用：list[数字] 按位置取值，dict["key"] 按 key 取值。

7. dict、键值对与 TOOL_HANDLER

TOOL_HANDLER = {} 表示创建一个空 dict。dict 里放 key-value 键值对。

```
TOOL_HANDLER = {
    "read_file": run_read_file,
    "write_file": run_write_file,
    "bash": run_bash,
}

handler = TOOL_HANDLER[block.name]
result = handler(**block.input)
```

规则	说明
{}	空字典，不是空集合。空集合要写 set()。
key 不要求都是字符串	可以是 str/int/float/bool/tuple/None 等。
key 必须可哈希	list/dict/set 一般不能做 key。
value 几乎任意	可以是字符串、数字、list、dict、函数、对象。
key 唯一	重复 key 会覆盖旧 value。

在 TOOL_HANDLER 里，key 通常是工具名字符串，value 是具体处理函数。它本质上是“工具名 -> 工具实现函数”的分发表。

8. 函数参数体系：形参、实参、位置参数、关键字参数

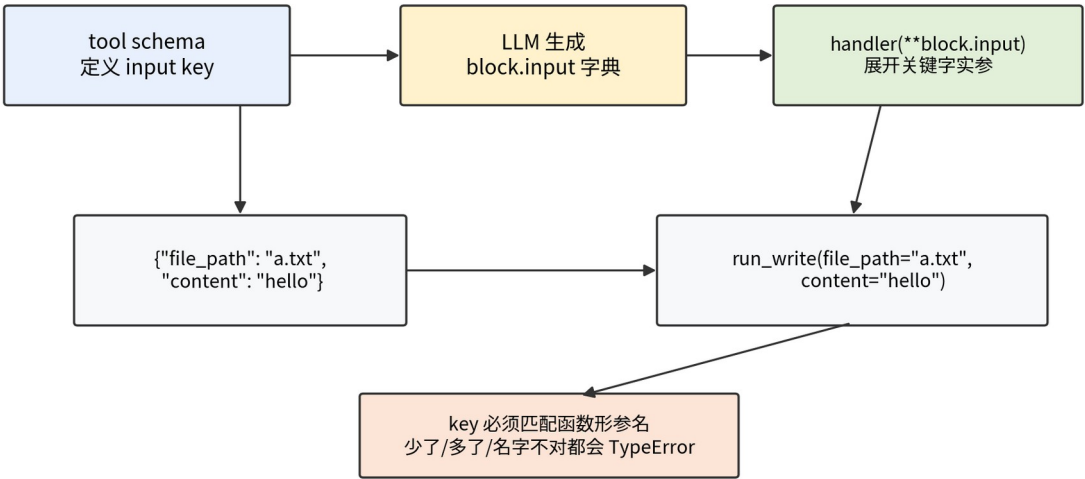
名称	出现位置	例子	含义
形参 parameter	函数定义时	def f(a, b):	接收外部传值的变量名。
实参 argument	函数调用时	f(1, 2)	真正传进去的值。
位置参数	调用时按顺序传	f(1, 2)	第 1 个值给第 1 个形参。
关键字参数	调用时写名字	f(a=1, b=2)	按名字匹配，不依赖顺序。
默认参数	定义时带默认值	def f(a, b=10):	不传时使用默认值。
*args	定义时收集多余位置参数	def f(*args):	收集成 tuple。
kwargs	定义时收集多余关键字参数	def f(kwargs):	收集成 dict。
keyword-only	* 后面的参数	def f(a, *, b):	b 必须用关键字传。
positional-only	/ 前面的参数	def f(a, /):	a 只能按位置传。

```
def example(a, b=10, *args, c, d=20, **kwargs):
    print(a, b, args, c, d, kwargs)

example(1, 2, 3, 4, c=5, d=6, x=7, y=8)
# a = 1
# b = 2
# args = (3, 4)
# c = 5
# d = 6
# kwargs = {"x": 7, "y": 8}
```

9. handler(**block.input) 的本质

重点： ****block.input** 会把 input 字典展开成关键字实参。字典的 key 会变成函数调用时的参数名，因此必须匹配工具实现函数的形参名。



```
def run_write(file_path: str, content: str) -> str:
    ...

block.input = {
    "file_path": "a.txt",
    "content": "hello"
}

run_write(**block.input)
# 等价于：
run_write(file_path="a.txt", content="hello")
```

情况	示例	结果
key 正好匹配	<code>{"file_path": "a.txt", "content": "hello"}</code>	正常调用。
key 少了	<code>{"file_path": "a.txt"}</code>	<code>TypeError: missing required</code>

		argument。
key 多了	{"file_path": "a.txt", "content": "hello", "mode": "append"}	TypeError: unexpected keyword argument。
key 名字不对	{"path": "a.txt", "content": "hello"}	如果函数形参叫 file_path，会报 unexpected keyword argument path。
函数有 **kwargs	def f(file_path, content, **kwargs)	多余 key 会被 kwargs 接住。

完整链路：tool schema 定义 input key -> LLM 生成 block.input 字典 -> **block.input 展开为关键字实参 -> Python 按形参名匹配 -> 执行工具函数。

10. 总体思维导图

下面是本次对话的总结构，用来把 Agent 机制和 Python 语法连接起来：

```
Agent 工具调用机制
|
+-- 架构层
|   +-- 主 agent：拆任务、调度、汇总
|   +-- 子 agent：独立完成明确子任务
|   +-- LLM API：可共用，但 history 必须隔离
|
+-- 消息层
|   +-- messages: list
|   |   +-- message: dict {role, content}
|   |       +-- content: str 或 block list
|   |           +-- TextBlock: type="text", text="..."
|   |           +-- ToolUseBlock: type="tool_use", name, input
|
+-- Python 数据结构层
|   +-- list[数字]：按位置取元素
|   +-- dict["key"]：按 key 取 value
|   +-- for x in list：每次拿一个列表元素
|   +-- for k in dict：默认每次拿一个 key
|
+-- 工具执行层
|   +-- TOOL_HANDLER: {工具名: 函数对象}
|   +-- block.name 找工具名
|   +-- block.input 是输入字典
|   +-- handler(**block.input) 展开为关键字实参
|
+-- 结果收敛层
    +-- 先看 messages[-1]["content"]
    +-- 如果不是文本，倒序找最近 assistant 文本
    +-- 最终只返回 result 摘要主 agent
```

11. 最容易混淆的点

易混点	正确理解
msg 是 dict, 为什么用 []?	dict 就是用 dict["key"] 按 key 取 value。
getattr 和 [] 的区别?	getattr 用于对象属性, 如 b.type; [] 用于字典 key, 如 msg["role"]。
for 循环是否按 schema 最小单位遍历?	不是。for 按 iterable 每次产出的元素遍历。list 每次产出一个元素。
schema 是否强制函数形参?	schema 引导 LLM 生成 input; 真正强制匹配的是 Python 的 **dict 关键字调用规则。
并发和并行是否一样?	并发是请求时间重叠; 并行是真正同时计算, 通常是供应商内部实现。
子 agent 是否应返回完整 history?	一般不返回。只返回摘要, 避免污染主 agent 上下文并降低 token 成本。

12. 推荐代码风格

- schema 的 input key 与工具函数形参名保持一致。
- handler(**block.input) 外层加 try/except, 避免工具参数错误直接炸掉主循环。
- 子 agent 和主 agent 使用独立 messages, 不要多个 agent 共享同一个 history 列表。
- 子 agent 返回结构化摘要, 而不是完整消息历史。
- 先限制子 agent 不再派遣 agent; 工程稳定后再加 max_depth、max_children、budget、timeout。
- 变量名统一, 例如全部用 result, 不要 results/result 混用。

最终记忆句: LLM 输出的是结构化工具调用; Python 用 dict、for、[]、getattr、**kwargs 把它转化为真实函数调用; agent loop 再把工具结果写回消息历史, 直到模型停止调用工具。